PROCESSAMENTO
DIGITAL DE IMAGENS

# OCR para Leitura de Placas Veiculares

Sistema de reconhecimento de caracteres em imagens de placas para condominios— carros e motos

Grupo: Caio Nunes · Edson Felipe · Nicolas Vidal
Disciplina: Processamento Digital de Imagens · [2026/Q1]

[ ABC-1D23 ]

OCR ATIVO

# O Problema: Leitura Automática de Placas

### 🚗 Segurança Condominial

Necessidade de identificar automaticamente veículos que entram e saem, aumentando o controle e reduzindo riscos de acesso não autorizado.

### 🅿 Controle de Acesso

Automatização de portarias e garagens, permitindo liberação rápida para moradores cadastrados e registro de visitantes sem depender exclusivamente de intervenção humana.

### 🔍 Rastreamento

Manter histórico de entradas e saídas de veículos, auxiliando na análise de ocorrências, aumento da segurança e apoio em casos de incidentes.

## O que é OCR?

Optical Character Recognition (Reconhecimento Óptico de Caracteres) é a tecnologia que converte imagens de texto em dados legíveis por máquina.

`Imagem → Pré-proc. → Segmentação → Texto`

### 💡 Por que é difícil?

Variação de iluminação, ângulo, sujeira na placa, fontes diferentes entre estados/países e baixa resolução tornam o problema desafiador.

# O que nos propusemos a construir

Desenvolver um sistema em Python capaz de capturar imagens de veículos via webcam, detectar a placa e retornar o texto reconhecido — compatível com Linux e Windows.

**01** **Captura via webcam**
Integração com câmera do dispositivo em tempo real

**02** **Pré-processamento**
Normalização de iluminação e contraste da imagem

**03** **Detecção da placa**
Localização do ROI (Região de Interesse) na imagem

**04** **Segmentação**
Isolamento de cada caractere da placa

**05** **OCR com Tesseract**
Reconhecimento do texto usando motor OCR open-source

**06** **Compatibilidade**
Funcional em Ubuntu/Linux e Windows sem alterações

# Stack utilizada no projeto

### Python 3

**Linguagem principal**

Ambiente virtual, pip, compatível com Linux e Windows

### OpenCV

**Processamento de imagem**

Leitura de câmera, morfologia, binarização, contornos

### NumPy

**Manipulação de arrays**

Operações matriciais sobre pixels e kernels

### Tesseract OCR

**Reconhecimento de texto**

Motor open-source de OCR via pytesseract (wrapper Python)

### Matplotlib

**Visualização e debug**

Exibição de etapas intermediárias do pipeline

### Linux & Windows

**Compatibilidade**

shutil.which() para encontrar tesseract em qualquer OS

# Fluxo geral do sistema

Captura de Imagem

Pré-processamento

Detecção da Placa

Segmentação de Chars.

OCR & Resultado

### Câmera / Arquivo

Webcam ao vivo ou imagens .jpg/.png de uma pasta. O usuário pressiona um botão para capturar.

### Resultado Final

Texto da placa impresso no terminal e exibido na janela. Ex: 'Placa OCR: ABC 1D23'

# Das etapas técnicas ao texto final

## 1. Pré-processamento

- Conversão para escala de cinza (cv2.cvtColor)
- Blur Gaussiano 5×5 (reduz ruído)
- Equalização de histograma (contraste)
- Operação Blackhat morfológica (realça texto escuro)

## 2. Detecção da Placa

- Binarização OTSU + inversão
- Fechamento morfológico (MORPH_CLOSE)
- Dilatação e Abertura
- Busca de contornos (findContours)
- Filtro: proporção e área do bounding rect

## 3. OCR dos Caracteres

- Extração do ROI com padding
- Binarização + isolamento de chars por contorno
- Separação em linha superior / inferior
- Redimensionamento 40×60 px por caractere
- Tesseract PSM 7 com whitelist alfanumérica

# Da captura ao resultado — passo a passo

**Imagem Original**
BGR colorido

→

**Blackhat (cinza)**
Texto realçado

→

**Binarização OTSU**
P&B invertido

→

**Fechamento Morfológico**
Regiões unidas

→

**Placa Isolada (ROI)**
Recorte padded

```
# Trecho do código: processamento e detecção


def processar_imagem(img_original):
    img_cinza = cv2.cvtColor(img_original, cv2.COLOR_BGR2GRAY)
    img_blur  = cv2.GaussianBlur(img_cinza, (5,5), 0)
    img_eq    = cv2.equalizeHist(img_blur)
    kernel    = np.ones((15,15), np.uint8)
    blackhat  = cv2.morphologyEx(img_eq, cv2.MORPH_BLACKHAT, kernel)
    return blackhat  # texto escuro realçado
```

# Obstáculos encontrados no desenvolvimento

## Placas de Moto

Motos têm duas linhas de texto (letras em cima, números embaixo). A proporção da placa é diferente, e o tamanho é menor — o que exigiu lógica de separação de linhas no código.

## ↗ Placas Inclinadas / Tortas

Imagens capturadas em ângulo causam deformação nos caracteres. Técnicas de perspectiva (warp) ajudam, mas aumentam a complexidade. No sistema atual, a binarização robusta ameniza o problema.

## 5/S B/8 Caracteres Semelhantes

O OCR frequentemente confunde S↔5, B↔8, O↔0, I↔1. A whitelist no Tesseract ajuda, mas não elimina completamente o erro — especialmente com placas sujas ou de baixa resolução.

## 💡 Iluminação Variável

Luz solar direta, reflexo no capô e sombras criam inconsistências. A equalização de histograma e o Blackhat morfológico foram cruciais para tornar o sistema mais robusto.

# O que funcionou e o que aprendemos

## Soluções Implementadas

Blackhat morfológico → Realce robusto do texto da placa independente da iluminação

Separação de linhas (sup/inf) → Suporte completo a placas de moto

Padding no ROI (+20% largura, +35% altura) → Evita cortar bordas da placa

Whitelist Tesseract (A–Z, 0–9) → Reduz confusões com símbolos irrelevantes

shutil.which() → Localiza o Tesseract automaticamente em Linux ou Windows

## Resultados Obtidos

**~70–80%**

Taxa de leitura correta (condições ideais)

**< 1s**

Tempo de processamento por imagem

⚠ Limitação: desempenho cai com placas sujas, ângulo >15° ou baixa resolução

Obrigado! Perguntas?

# O que construímos e para onde ir

Desenvolvemos um sistema funcional de OCR para leitura de placas, cobrindo todo o pipeline de PDI — do pré-processamento à extração de texto — com suporte a carros e motos em Linux e Windows.

## O que aprendemos

- → Morfologia matemática é chave para realce de texto
- → OCR genérico falha sem pré-processamento adequado
- → Portabilidade exige atenção desde o início do projeto
- → Separação de casos (carro vs. moto) melhora a precisão

## Melhorias futuras

- → Correção de perspectiva (warp) para placas inclinadas
- → Redes neurais (YOLOv8) para detecção mais robusta
- → Interface gráfica (GUI) com feedback visual em tempo real
- → Pós-processamento: validar formato Mercosul (XXX0X00)